
RAPPORT DE CONCEPTION

FlotteManager

Application web de gestion de flotte de véhicules

Table des matières

1. Introduction et contexte
2. Architecture technique
3. Diagramme de classes UML
4. Concepts orientés objet — justifications
5. Modules fonctionnels
6. Gestion de la persistance
7. Architecture MVC — Servlet / JSP
8. Difficultés rencontrées et solutions
9. Répartition du travail
10. Conclusion

1. Introduction et contexte

FlotteManager est une application web Java permettant à une entreprise de transport de gérer l'ensemble de sa flotte de véhicules : missions, chauffeurs, incidents et maintenance. Elle a été développée dans le cadre d'un projet universitaire imposant l'utilisation avancée des mécanismes orientés objet de Java.

Le projet couvre un domaine métier réaliste : planification de missions, affectation de chauffeurs et de véhicules, suivi des incidents, et visualisation statistique de l'activité.

Périmètre fonctionnel

Module	Fonctionnalités principales
Missions	Création, affectation de chauffeur et véhicule, clôture, filtrage, tri
Chauffeurs	CRUD, disponibilité en temps réel, historique missions
Véhicules	CRUD, 3 types (Léger, Lourd, Spécial), filtrage, statistiques
Incidents	Déclaration (Panne / Accident), traitement, filtrage multicritères
Statistiques	Tableau de bord global avec graphiques interactifs (Chart.js)

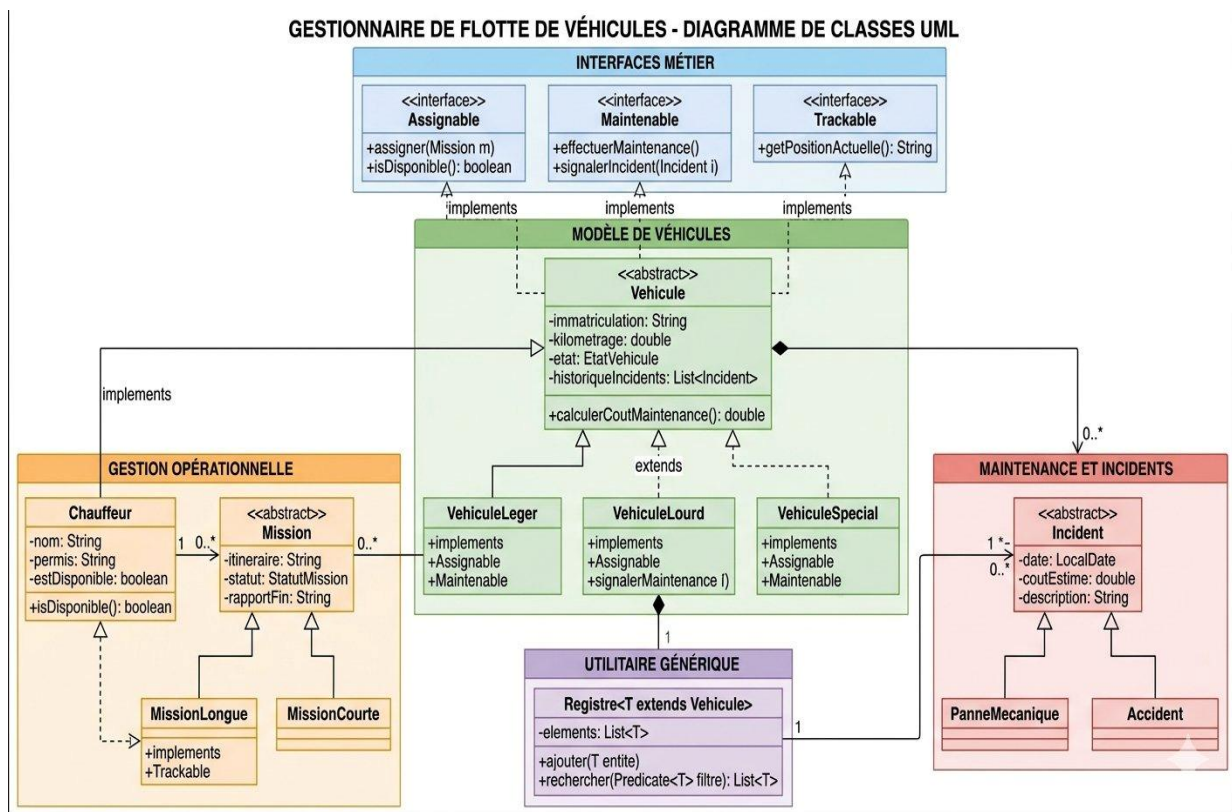
2. Architecture technique

Composant	Technologie / Rôle
Langage	Java 11 — Logique métier et contrôleurs
Serveur	Apache Tomcat 7 (Maven plugin) — Exécution des Servlets
Vue	JSP + JSTL + EL — Rendu HTML côté serveur
UI	Bootstrap 5 — Interface responsive et cohérente
Graphiques	Chart.js — Visualisations statistiques
Build	Maven 3 — Gestion des dépendances et déploiement
Persistance	Sérialisation Java — Sauvegarde des données dans des fichiers .dat
Versioning	Git / GitHub — Travail collaboratif en équipe de 4

Structure du projet

```
src/main/
├── java/fr/flotte/
│   ├── model/      ← Classes métier (Mission, Vehicule, Incident, Chauffeur...)
│   ├── controller/ ← Servlets (MissionServlet, VehiculeServlet, IncidentServlet...)
│   ├── exception/  ← Exceptions métier personnalisées
│   └── util/        ← PersistenceUtil (sérialisation / désérialisation)
└── webapp/
    ├── assets/      ← Images et vidéos de la page d'accueil
    └── *.jsp         ← Pages web (missions, chauffeurs, véhicules, incidents...)
```

3. Diagramme de classes UML



4. Concepts orientés objet — justifications

4.1 Classes abstraites (3)

Mission — abstract

Toutes les missions partagent un itinéraire, un statut, une date de début et un identifiant unique. La méthode `genererRapport()` est abstraite car son contenu dépend du type de mission (courte ou longue). On impose ainsi à chaque sous-classe de produire un rapport adapté à sa nature.

```
public abstract class Mission implements Serializable {  
    public abstract String genererRapport();  
    public abstract String getType();  
}
```

Vehicule — abstract

Léger, Lourd et Spécial partagent immatriculation, marque, kilométrage et état. Les méthodes `estDisponible()` et `effectuerMaintenance()` sont abstraites car leur comportement est spécifique à chaque type de véhicule.

Incident — abstract

Panne mécanique et Accident partagent date, coût estimé et identifiant véhicule. Les méthodes `genererRapport()` et `getType()` sont abstraites car un accident inclut un niveau de gravité tandis qu'une panne précise le composant défaillant. On ne crée jamais un incident sans savoir s'il s'agit d'une panne ou d'un accident : l'instanciation directe est donc impossible et incorrecte dans le domaine métier.

4.2 Interfaces (3)

Interface	Méthodes imposées	Implémentée par — Justification
Assignable	<code>estDisponible(): boolean</code>	Chauffeur, Vehicule — Contrat commun entre deux entités très différentes (humain et machine) qui peuvent toutes deux être réservées ou libres.
Trackable	<code>getPositionActuelle(): String</code> <code>mettreAJourPosition(String)</code>	VehiculeSpecial — Seuls les véhicules spéciaux nécessitent un suivi GPS temps réel. L'interface isole cette capacité sans la forcer sur tous les véhicules.
Maintenable	<code>estEnMaintenance(): boolean</code> <code>signalerIncident(Incident)</code> <code>getHistoriqueIncidents(): List</code>	VehiculeLeger, VehiculeLourd, VehiculeSpecial — Tous les véhicules peuvent subir des incidents et passer en maintenance. L'interface définit ce contrat sans le placer dans la classe abstraite Vehicule.

4.3 Génériques bornés

Deux gestionnaires utilisent des génériques bornés afin de rester réutilisables tout en garantissant la sécurité de type à la compilation :

```
// Gestionnaire opérationnel : accepte n'importe quelle sous-classe de Mission  
public class GestionnaireOperationnel<T extends Mission> implements Serializable {  
    private List<T> missions;  
    private Map<String, Chauffeur> chauffeurs;  
    private transient PriorityQueue<T> filePrioritaire;  
}
```

```
// Registre des véhicules : accepte n'importe quelle sous-classe de Vehicule
public class RegistreVehicule<T extends Vehicule> {
    private List<T> vehicules;
}
```

Avantage : si une nouvelle sous-classe `MissionUrgence` extends `Mission` est créée plus tard, `GestionnaireOperationnel<MissionUrgence>` fonctionnera immédiatement sans aucune modification du gestionnaire.

4.4 Pattern Singleton

Les trois gestionnaires principaux sont des singletons synchronisés afin que l'état des données soit unique et cohérent entre tous les appels HTTP, même en accès concurrent :

Singleton	Données gérées
GestionnaireOperationnel	Missions et chauffeurs
RegistreVehicule	Flotte de véhicules
GestionnaireMaintenance	Incidents

```
public static synchronized GestionnaireOperationnel<Mission> getInstance() {
    if (instance == null) {
        instance = new GestionnaireOperationnel<>();
    }
    return instance;
}
```

4.5 Collections (3 types)

Collection	Usage	Justification
List<T> (ArrayList)	Missions, véhicules, incidents	Accès indexé, ajout/suppression en fin de liste, itération ordonnée.
Map<String, Chauffeur> (LinkedHashMap)	Chauffeurs	Recherche en O(1) par identifiant unique, ordre d'insertion préservé.
PriorityQueue<T>	File de missions prioritaires	Les missions longues sont traitées en priorité via un comparateur personnalisé. Structure adaptée à une file d'attente triée.

4.6 Streams et Lambdas

Les Streams Java 8 sont utilisés pour le filtrage, le tri et les statistiques, en remplacement de toutes les boucles for classiques :

```
// Filtrage multicritères avec Predicate composé
public List<T> filtrerMulticriteres(String statut, String type, String chauffeurId) {
    Predicate<T> pred = m -> true;
    if (statut != null && !statut.isEmpty())
        pred = pred.and(m -> statut.equals(m.getStatut()));
    if (type != null && !type.isEmpty())
        pred = pred.and(m -> type.equals(m.getType()));
    return missions.stream().filter(pred).collect(Collectors.toList());
}
```

```
// Tri dynamique avec Comparator
public List<T> trierPar(String colonne, boolean croissant) {
    Comparator<T> comp = (a, b) -> a.getStatut().compareTo(b.getStatut());
    return missions.stream()
```

```

        .sorted(croissant ? comp : comp.reversed())
        .collect(Collectors.toList());
    }

```

```

// Statistiques avec groupingBy
public Map<String, Long> statistiquesParType() {
    return incidents.stream()
        .collect(Collectors.groupingBy(Incident::getType, Collectors.counting()));
}

```

4.7 Exceptions métier personnalisées (5)

Exception	Déclenchée quand
ChauffeurIndisponibleException	Affectation d'un chauffeur déjà en mission
MissionDejaTermineeException	Tentative de modification d'une mission clôturée
IncidentDejaTraiteException	Double traitement d'un incident déjà résolu
VehiculeIndisponibleException	Affectation d'un véhicule non disponible
CapaciteDepasseeException	Dépassement de la capacité de charge d'un véhicule lourd

```

public void traiterIncident(String id) throws IncidentDejaTraiteException {
    Incident inc = trouverIncident(id)
        .orElseThrow(() -> new IllegalArgumentException("Incident introuvable"));
    if (inc.isTraite()) throw new IncidentDejaTraiteException(id);
    inc.setTraite(true);
}

```

5. Modules fonctionnels

5.1 Gestion des missions

- Créer une MissionCourte (itinéraire simple) ou une MissionLongue (avec nombre de pauses)
- Affecter un chauffeur disponible — lève ChauffeurIndisponibleException sinon
- Affecter un véhicule disponible — le véhicule passe à l'état UTILISE
- Terminer une mission — le chauffeur et le véhicule repassent en DISPONIBLE
- Filtrer par statut / type, trier par date / itinéraire / statut

5.2 Gestion des chauffeurs

- CRUD complet avec modal de modification Bootstrap
- Disponibilité gérée via l'interface Assignable
- Historique des missions par chauffeur

5.3 Gestion des véhicules

- VehiculeLeger : nombre de places, consommation
- VehiculeLourd : capacité de charge, nombre d'essieux
- VehiculeSpecial : type spécial, position GPS, mode urgence — implémente Trackable
- Filtrage par marque, état, kilométrage max ; tri multi-colonnes
- Page statistiques dédiée : véhicule le plus kilométré, répartition par état, km moyen

5.4 Gestion des incidents et maintenance

- PanneMecanique : type de panne (moteur, freins, pneu, électrique, autre)
- Accident : niveau de gravité (léger / grave)
- Marquer un incident comme traité — lève IncidentDejaTraiteException si déjà résolu
- Filtrer par type, statut (traité / non traité), véhicule concerné
- Trier par date, type, coût estimé

5.5 Tableau de bord statistique

- Missions : total, en cours, terminées, courtes vs longues (graphiques camembert et barres)
- Chauffeurs : disponibles / occupés
- Véhicules : disponibles / en mission / en maintenance, répartition par marque, km moyen
- Incidents : total, traités / non traités, coût total et moyen, répartition par type

6. Gestion de la persistance

La persistance est assurée par la sérialisation Java via la classe utilitaire `PersistanceUtil`. Trois fichiers `.dat` distincts stockent les données entre les sessions du serveur :

Fichier	Contenu
<code>flotte_operationnel.dat</code>	Missions et chauffeurs
<code>flotte_vehicules.dat</code>	Registre des véhicules
<code>flotte_maintenance.dat</code>	Incidents

Les singletons sont restaurés au démarrage du serveur via `charger*()`, puis sauvegardés après chaque opération d'écriture via `sauvegarder*()`. La classe `PriorityQueue` est marquée `transient` et reconstruite à la désérialisation via `readObject()`.

7. Architecture MVC — Servlet / JSP

L'application suit strictement le patron MVC :

Couche	Rôle	Exemples
Modèle (M)	Logique métier, données	Mission, Chauffeur, GestionnaireOperationnel
Vue (V)	Présentation HTML, aucune logique	missions.jsp, chauffeurs.jsp, incidents.jsp
Contrôleur (C)	Traitement des requêtes HTTP, coordination	MissionServlet, ChauffeurServlet, IncidentServlet

Flux typique : le navigateur envoie une requête GET ou POST → la Servlet interprète l'action, appelle le gestionnaire métier, place les résultats dans `request.setAttribute()` → forward vers la JSP qui affiche les données via JSTL/EL.

Routes applicatives

URL	Servlet → Page JSP
/home	HomeServlet → accueil.jsp
/missions	MissionServlet → missions.jsp, detail-mission.jsp
/chauffeurs	ChauffeurServlet → chauffeurs.jsp
/vehicules	VehiculeServlet → vehicules_liste.jsp, vehicules_formulaire.jsp, vehicules_detail.jsp, vehicules_statistique.jsp
/incidents	IncidentServlet → incidents.jsp
/statistiques	StatistiquesServlet → statistiques.jsp

8. Difficultés rencontrées et solutions

Difficulté 1 — Conflits Git lors du merge en équipe

Deux membres ayant modifié simultanément les mêmes fichiers JSP (navbars), le git pull a généré des conflits dans 5 fichiers.

Solution : résolution manuelle des marqueurs de conflit en conservant la version HEAD (liens de navigation corrects avec accents et classe active bien placée), puis commit de merge et push.

Difficulté 2 — Partage du singleton GestionnaireMaintenance

L'IncidentServlet et la StatistiquesServlet doivent accéder au même gestionnaire d'incidents. Un simple new dans chaque Servlet crée des instances séparées — les incidents déclarés n'apparaissent pas dans les statistiques.

Solution : pattern Singleton sur GestionnaireMaintenance avec une méthode statique setInstance() permettant à PersistanceUtil de restaurer l'état depuis le fichier .dat.

Difficulté 3 — Synchronisation état véhicule ↔ mission

Lors de l'affectation d'un véhicule à une mission, son état doit passer à UTILISE, puis revenir à DISPONIBLE à la clôture. Un identifiant objet direct rendrait la sérialisation circulaire.

Solution : stocker uniquement l'immatriculation (vehiculeId : String) dans la mission. La Servlet résout la référence via RegistreVehicule et met à jour l'état manuellement, évitant toute dépendance circulaire.

9. Répartition du travail

Membre	Module principal — Fichiers clés
Malika AMRANI	Gestion opérationnelle (missions + chauffeurs + statistiques) — Mission, MissionCourte/Longue, Chauffeur, GestionnaireOperationnel, MissionServlet, ChauffeurServlet, StatistiquesServlet, missions.jsp, chauffeurs.jsp, statistiques.jsp
Nirmala BONAMI	Gestion des véhicules — Vehicule, VehiculeLeger/Lourd/Spécial, RegistreVehicule, VehiculeServlet, vehicules_*.jsp
Sofia BOURAZA	Gestion des véhicules (co-développement) — EtatVehicule, Trackable, VehiculeException, CapaciteDepasseeException
Jennifer HILAIRE	Gestion des incidents et maintenance — Incident, PanneMecanique, Accident, GestionnaireMaintenance, Maintenable, IncidentServlet, incidents.jsp, IncidentDejaTraiteException

La page d'accueil (accueil.jsp), la persistance (PersistanceUtil), l'intégration des statistiques globales et la résolution des conflits Git ont été effectuées en collaboration par l'ensemble du groupe.

10. Conclusion

Ce projet nous a permis de mettre en pratique l'ensemble des mécanismes avancés de Java dans un contexte métier réaliste. La hiérarchie de classes (Mission, Vehicule, Incident) illustre concrètement l'intérêt des classes abstraites pour factoriser le comportement commun tout en laissant chaque sous-classe exprimer sa spécificité.

Les interfaces (Assignable, Trackable, Maintenable) ont permis d'établir des contrats entre entités hétérogènes — chauffeurs et véhicules — sans créer de couplage fort.

Le travail en équipe via Git, avec branches et pull requests, nous a confrontées à des situations concrètes de développement collaboratif (conflits de merge, coordination des singletons partagés) et nous a permis d'adopter des pratiques professionnelles.

L'architecture MVC Servlet/JSP nous a donné une vision claire de la séparation des responsabilités, socle de frameworks modernes comme Spring MVC.